

FinVista Capital – Financial Intelligence Assistant

1. Project Overview:

1.1. Purpose

The FinVista Capital-Financial Intelligence Assistant is an enterprise-grade AI solution designed to revolutionize how financial professionals interact with complex documentation. By leveraging a Retrieval-Augmented Generation (RAG) architecture, this application allows users to query large datasets, such as annual reports, market research, and compliance documents, using natural language to receive accurate, context-aware insights.

1.2. Key Objectives

- **Contextual Intelligence:** Move beyond simple keyword searches by providing semantic understanding of financial documents.
- **Grounded Accuracy:** Eliminate hallucinations by enforcing source-based responses with direct citations (document name and page number).
- **Scalable Architecture:** Utilize containerization and cloud-native Kubernetes deployment to ensure high availability and elastic performance.
- **Production Readiness:** Integrate DevOps best practices through automated deployment workflows, secure configuration management, and optimized resource utilization.

2. Architecture and System Design:

2.1. High-Level Architecture

The FinVista Capital - Financial Intelligence Assistant utilizes a cloud-native, microservices-inspired architecture designed for modularity and scalability. The system flow is as follows:

- **User Interface:** A responsive **Streamlit** dashboard provides a conversational interface for users to submit natural language queries.
- **Intelligence Engine:** The backend leverages a **Retrieval-Augmented Generation (RAG)** pipeline to fetch relevant data from uploaded financial documents and process it via the Gemini API.
- **Containerization:** The entire application stack is containerized using **Docker**, ensuring consistency across development, testing, and production environments.
- **Cloud Orchestration:** The containerized application is deployed on **Amazon Elastic Kubernetes Service (EKS)**, which handles load balancing and high availability.
- **Infrastructure Management:** The deployment is managed via Kubernetes manifests (deployment.yaml and service.yaml) and orchestrated using eksctl and kubectl.

2.2. Data Flow

- **Ingestion:** Financial reports are uploaded and processed into vector embeddings for semantic search.
- **Query:** The user submits a question through the web interface.
- **Retrieval:** The system searches the vector store for context-relevant document segments.

- **Generation:** The LLM synthesizes an answer based on the retrieved context and presents it to the user with specific source citations.

3. Technical Implementation (Deployment):

This section details the migration of the application from a local environment to a cloud-native Kubernetes infrastructure.

3.1. Infrastructure Provisioning

The application is hosted on **Amazon EKS (Elastic Kubernetes Service)** to ensure scalability and reliability. The cluster was provisioned using `eksctl`, an official CLI tool for Amazon EKS.

- **Cluster Creation:** The command `eksctl create cluster --name <cluster-name> --region <region>` was utilized to provision the control plane and worker nodes.
- **Infrastructure as Code:** By utilizing `eksctl`, the environment setup is repeatable and documented, adhering to industry-standard infrastructure-as-code practices.

3.2. Containerization and Orchestration

To ensure environmental consistency, the application was packaged into a Docker container and deployed using Kubernetes manifests.

- **Containerization:** A Dockerfile was authored to define the environment, install dependencies, and run the Streamlit application.
- **Amazon ECR Integration:** The application image was pushed to **Amazon Elastic Container Registry (ECR)**. This serves as a centralized, secure repository that allows the EKS cluster to pull the image during deployment.
- **Workflow:** The process involved authenticating the Docker client with ECR, tagging the local image, and pushing it to the private repository.
- **Deployment Configuration:** The `deployment.yaml` manifest defines the desired state, including the container image URI from Amazon ECR and necessary environment variables like `GOOGLE_API_KEY` for secure API authentication.
- **Networking:** A Kubernetes `service.yaml` was configured to expose the application to the internet via an **AWS LoadBalancer**, allowing end-users to interact with the assistant.

3.3. Deployment Verification

Verification was performed using `kubectl` commands to ensure all services were healthy and operational.

- **Pod Status:** `kubectl get pods` was used to confirm that the container reached the Running state.
- **Service Accessibility:** `kubectl get services` allowed for the retrieval of the EXTERNAL-IP required to access the application dashboard.

4. CI/CD Pipeline:

This section demonstrates how you automate the path from code changes to production.

- **Workflow Automation:** The project utilizes a CI/CD workflow to ensure that updates to the application are automatically built, tested, and deployed.

- **Version Control:** Git serves as the single source of truth for the codebase.
- **Continuous Deployment:** Any updates to the source code trigger the build process, leading to a new container image being pushed to Amazon ECR and updated in the EKS cluster.

5. Tools and Technologies:

The following toolchain was utilized to design, develop, and deploy the **FinVista Capital - Financial Intelligence Assistant**:

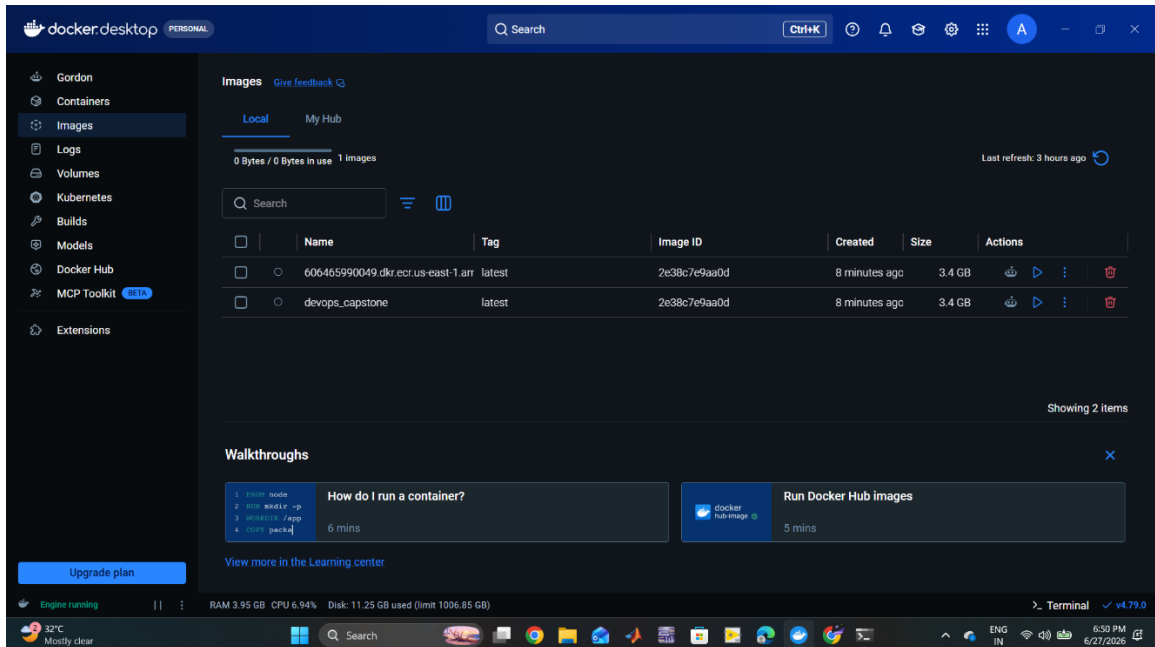
Category	Tools / Technologies
Programming Language	Python
AI / GenAI Framework	Gemini API, LangChain (for RAG pipeline)
Frontend Framework	Streamlit
Containerization	Docker
Container Registry	Amazon Elastic Container Registry (ECR)
Cloud Infrastructure	Amazon Web Services (AWS)
Orchestration	Amazon Elastic Kubernetes Service (EKS), eksctl, kubectl
Version Control	Git, GitHub

6. Technical Appendix (Implementation Commands):

This section lists the key commands used during the deployment phase, validating the **Kubernetes deployment** and **CI/CD automation** criteria.

6.1. Docker and ECR Workflow

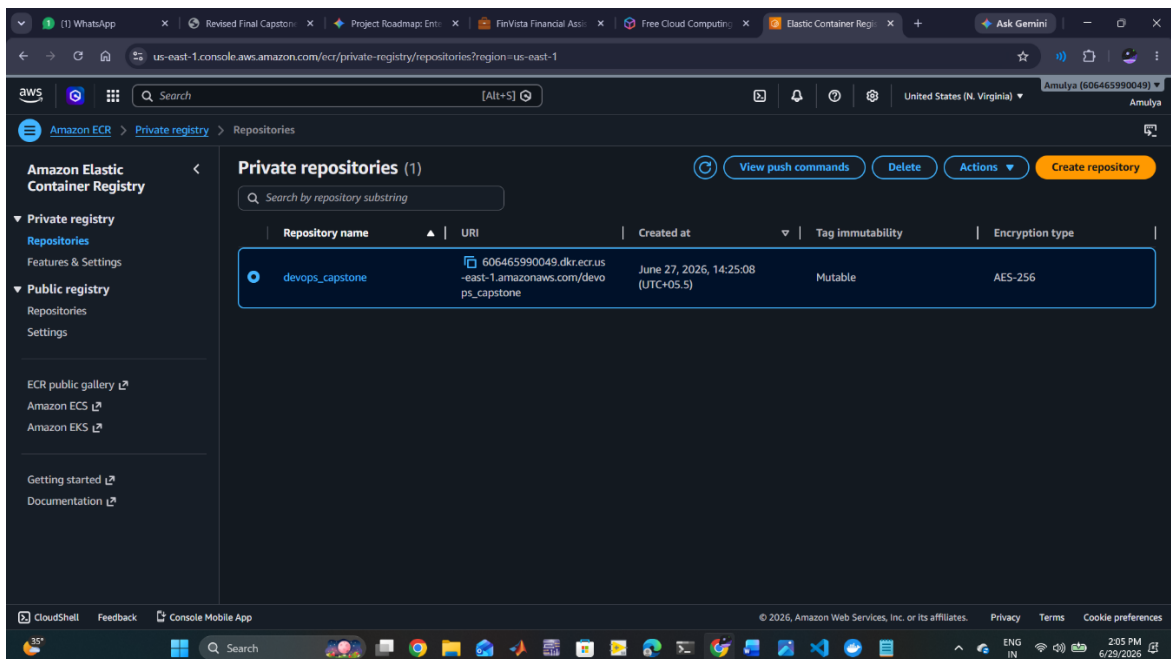
- **Build Image:**
docker build -t finvista-app .
Docker image created in **Docker Desktop**

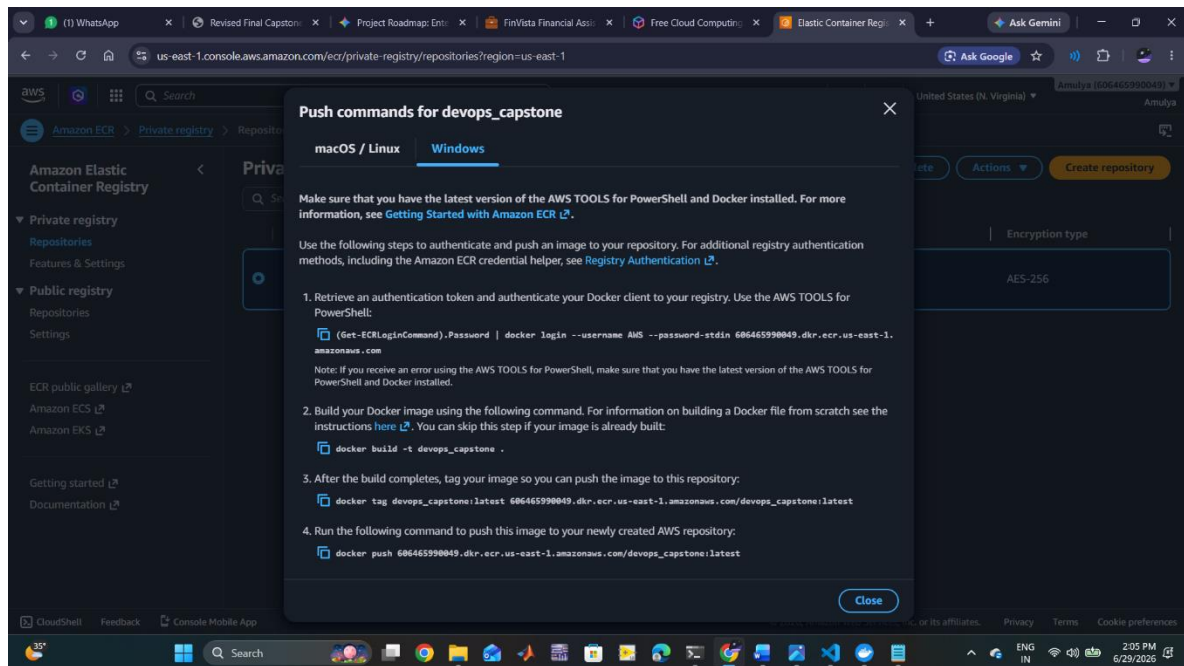


- **Tag and Push to ECR:**

`docker tag finvista-app:latest <aws_account_id>.dkr.ecr.<region>[.amazonaws.com/finvista-app:latest](https://.amazonaws.com/finvista-app:latest)`

`docker push <aws_account_id>.dkr.ecr.<region>[.amazonaws.com/finvista-app:latest](https://.amazonaws.com/finvista-app:latest)`





6.2. Infrastructure and Deployment

- **Create Cluster:**

eksctl create cluster --name finvista-cluster --region us-east-1 --nodes 2

```
Windows PowerShell
Run 'docker image COMMAND --help' for more information on a command.
PS C:\Users\HP\Financial-intelligence-assistant> docker images

IMAGE
606465990049.dkr.ecr.us-east-1.amazonaws.com/devops_capstone:latest
devops_capstone:latest
PS C:\Users\HP\Financial-intelligence-assistant> eksctl version
0.227.0
PS C:\Users\HP\Financial-intelligence-assistant> aws configure
AWS Access Key ID [*****4W3]:
AWS Secret Access Key [*****pFjE]:
Default region name [us-east-1]:
Default output format [None]:
PS C:\Users\HP\Financial-intelligence-assistant> eksctl create cluster `
>> --name financial-assistant-cluster `
>> --region us-east-1
>> --nodegroup-name standard-workers `
>> --node-type t3.medium `
>> --nodes 2 `
>> --managed
2026-06-29 12:09:23 [I] eksctl version 0.227.0
2026-06-29 12:09:23 [I] using region us-east-1
2026-06-29 12:09:24 [I] setting availability zones to [us-east-1c us-east-1a]
2026-06-29 12:09:24 [I] subnets for us-east-1c - public:192.168.0.0/19 private:192.168.64.0/19
2026-06-29 12:09:24 [I] subnets for us-east-1a - public:192.168.32.0/19 private:192.168.96.0/19
2026-06-29 12:09:24 [I] nodegroup "standard-workers" will use "" [AmazonLinux2023/1.34]
2026-06-29 12:09:24 [I] Auto Mode will be enabled by default in an upcoming release of eksctl. This means managed node groups and managed networking add-on
s will no longer be created by default. To maintain current behavior, explicitly set 'autoModeConfig.enabled: false' in your cluster configuration. Learn mo
re: https://eksctl.io/usage/auto-mode/
2026-06-29 12:09:24 [I] using Kubernetes version 1.34
2026-06-29 12:09:24 [I] creating EKS cluster "financial-assistant-cluster" in "us-east-1" region with managed nodes
2026-06-29 12:09:24 [I] will create 2 separate CloudFormation stacks for cluster itself and the initial managed nodegroup
2026-06-29 12:09:24 [I] if you encounter any issues, check CloudFormation console or try 'eksctl utils describe-stacks --region=us-east-1 --cluster=financi
al-assistant-cluster'
2026-06-29 12:09:24 [I] Kubernetes API endpoint access will use default of {publicAccess=true, privateAccess=false} for cluster "financial-assistant-cluste
r" in "us-east-1"
2026-06-29 12:09:24 [I] CloudWatch logging will not be enabled for cluster "financial-assistant-cluster" in "us-east-1"
2026-06-29 12:09:24 [I] you can enable it with 'eksctl utils update-cluster-logging --enable-types={SPECIFY-YOUR-LOG-TYPES-HERE (e.g. all)} --region=us-eas
t-1 --cluster=financial-assistant-cluster'
2026-06-29 12:09:24 [I] default addons vpc-cni, kube-proxy, coredns, metrics-server were not specified, will install them as EKS addons
```

- **Deploy Manifests:**

kubectl apply -f deployment.yaml

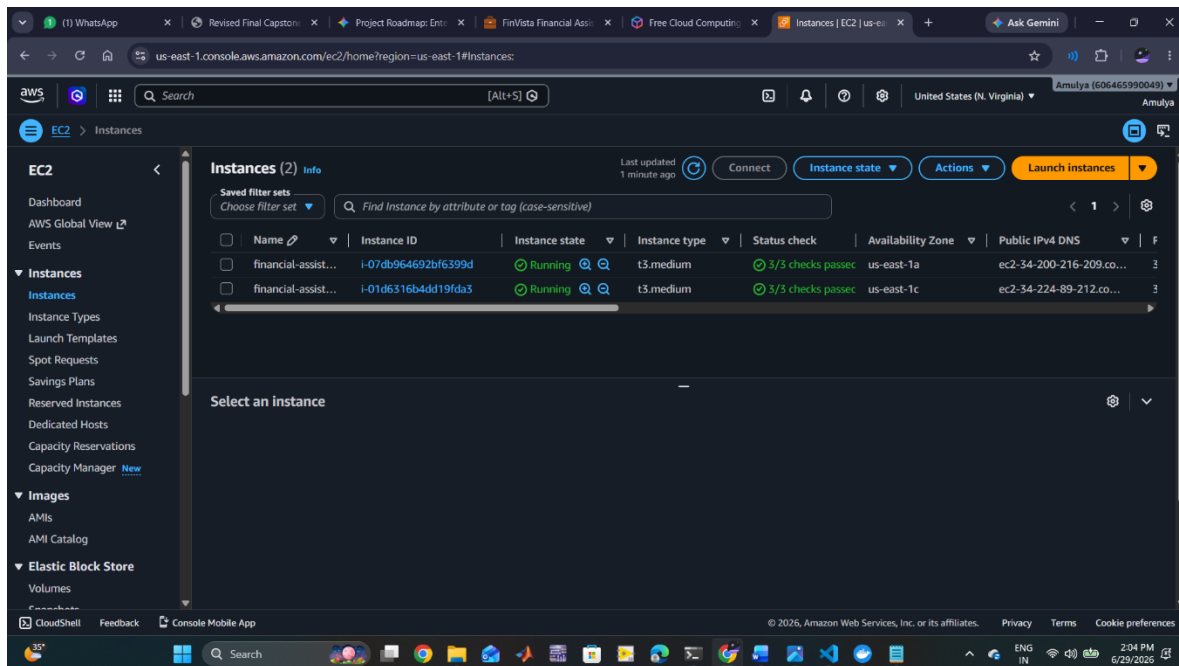
kubectl apply -f service.yaml

- **Verify Deployment:**

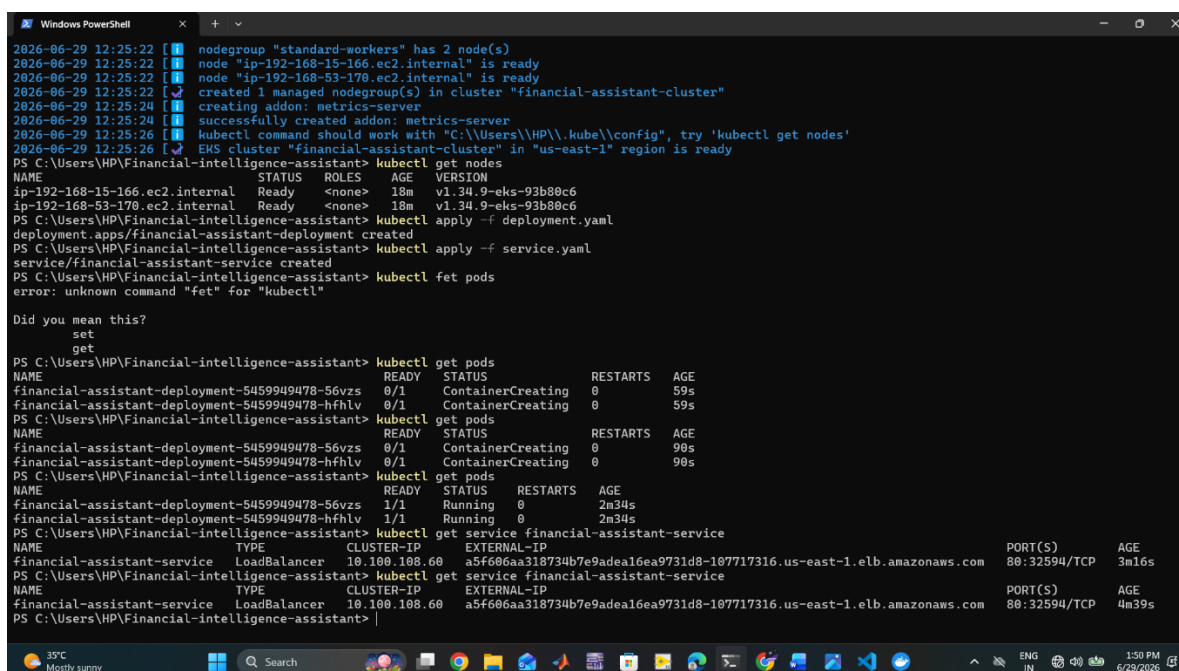
kubectl get pods

kubectl get svc

Pods are created



Pods are running

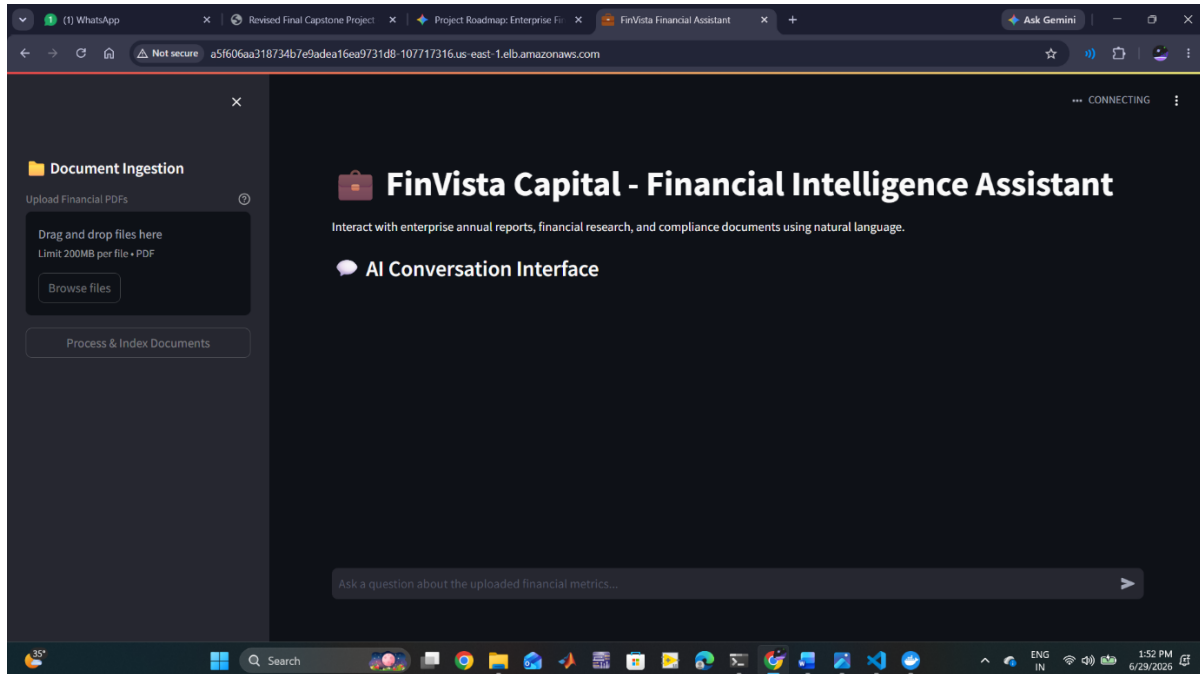


7. Output:

After get service financial-assistant-service

Copy the external link and paste it in the web browser

The RAG app will open using Streamlit interface



After opening upload the document related to the annual report and process & index the document

Prompt a question to it then it will give you the matched answers with the citations

